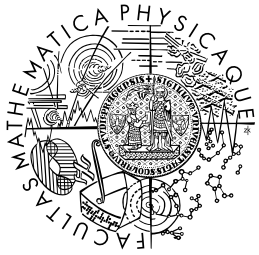

FACULTY
OF MATHEMATICS
AND PHYSICS
Charles University

BACHELOR THESIS

Adam Balko

Arduino platform simulator

Department of Software and Computer Science Education

Supervisor of the bachelor thesis: RNDr. David Bednárek, Ph.D.

Study programme: Computer Graphics, Vision and Computer Games Development

Prague 2026

I declare that I carried out this bachelor thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In ..... date ....

Author's signature

Dedication.

Title: Arduino platform simulator

Author: Adam Balko

Department: Department of Software and Computer Science Education

Supervisor: RNDr. David Bednárek, Ph.D., Department of Software Engineering

Abstract: Use the most precise, shortest sentences that state what problem the thesis addresses, how it is approached, pinpoint the exact result achieved, and describe the applications and significance of the results. Highlight anything novel that was discovered or improved by the thesis. Maximum length is 200 words, but try to fit into 120. Abstracts are often used for deciding if a reviewer will be suitable for the thesis; a well-written abstract thus increases the probability of getting a reviewer who will like the thesis.

Keywords: digital circuit simulator, Arduino

Název práce: Simulátor prostředí Arduino

Autor: Adam Balko

Katedra: Katedra softwaru a výuky informatiky

Vedoucí bakalářské práce: RNDr. David Bednárek, Ph.D., Katedra softwarového inženýrství

Abstrakt: Abstrakt práce přeložte také do češtiny.

Klíčová slova: simulátor číslicových obvodů, Arduino

Contents

Introduction

Arduino is a very popular microcomputer among amateurs and professionals alike. Despite this, testing the code for an Arduino machine can be fairly difficult. Debugging options are limited and the user needs to rely on additional measuring equipment. The usual choice is to test the code with an AVR emulator. Emulation is the process of running machine code on a hardware that doesn't match its instruction set. AVR is the processor that controls the Arduino board. These emulators are sometimes part of a bigger logical circuit simulation system, other times they are just a single piece of software.

Goal of this work is to skip the emulation process and create a working Arduino simulator by mimicking the changes of digital values on the board's pinout. Bundled with this simulator is a graphic application for creating and testing logical circuits, with Arduino as it's main component. The point of this application is to attach to the running Arduino simulator and visualize the expected Arduino behavior with user's code on a given circuit.

0.1 Arduino programming guide

0.2 Comparison of existing solutions

0.3 Specific goals of the thesis

1 Architecture design

1.1 Architecture overview

The whole project is divided into two main parts. The purpose of the backend is to execute the Arduino code and simulate the change on the board's pinout. The purpose of the frontend is to reflect these changes on a user defined logical circuit, while offering the user control over the simulation. We chose to implement the backend in C++ and the frontend in C#. Both backend and frontend are running in its own separate process.

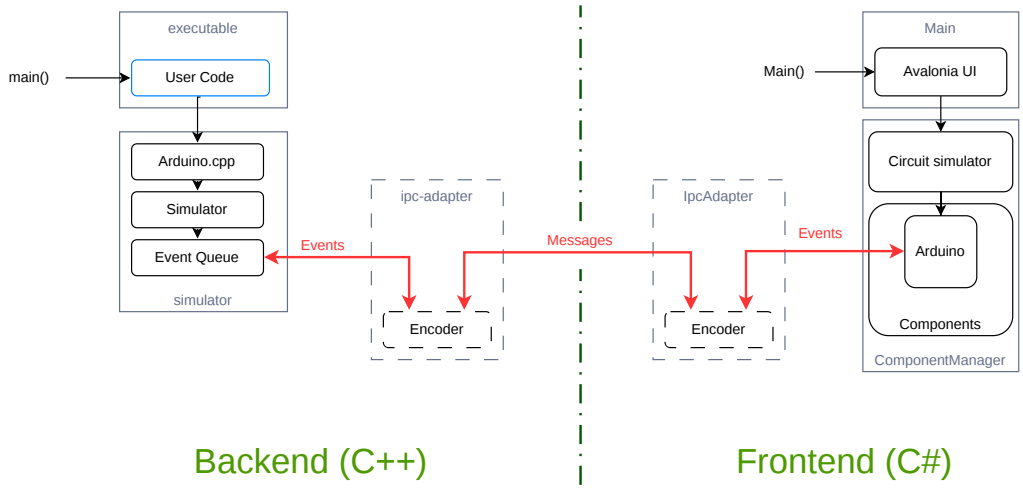

Figure 1.1 Top-level architecture overview

The Figure 1.1 shows the most important systems in each process. The user coded is compiled together with the `executable` module of the backend. The declarations in the header `Arduino.h` prepended to the top of the `.ino` is implemented by a mock library `Arduino.cpp`. For each observable action an event is created and passed to the singleton `Simulator` class to compute and put into an event queue. The event is also communicated to the frontend process.

In the frontend the events are consumed by the `Arduino` component, one of several components predefined as part of the `ComponentManager` project. This project is responsible for loading user-defined components and scenes, which connect the components together in a circuit. The circuit is simulated in the same project. The user interface is implemented in the `Main` project using Avalonia UI as a framework. Interacting with the UI may invoke new events in the `Arduino` component. Those are sent back to the backend's event queue in the same manner as they were received.

The IPC Adapter is an interface implemented in each process responsible for encoding and decoding the events into a standardised format and relaying to the other process.

1.2 Inter-process communication

Implementing processes that depend on each other comes with certain challenges. Processes always run concurrently, so a variety of synchronization techniques must be applied to avoid race conditions. Before that we need to determine how will the processes exchange data. Process is a kernel level mechanism, thus approaches to handle the exchange can vary between operating systems. These mechanisms are what is called *Inter-process communication*, or IPC for short.

1.2.1 Considerations

There is a technical and an engineering reason for splitting the work into more than one process.

The user must be able to have full control over the frontend user interface at all times, even while the Arduino code is being debugged. Every common debugger blocks all running threads of a process, when a breakpoint is hit. If the GUI has been part of the same process as the systems running the simulation, it would freeze and become unresponsive every time a breakpoint is hit. The only way to avoid this behavior is to separate the GUI to a different process. We want to support breakpoints in our project, as they are the most useful tool in a debugger's toolkit.

Splitting the application into multiple processes is actually a common practice among larger pieces of software. The program of each process can be written in a different programming language and thus delegated into teams with different competencies. They mark clear separation of concerns and force a solid interface design. Each process is easily swappable for another implementation with the same interface while the other processes are still running. In our case, the GUI process could be replaced by a web client, a CLI client, or an IDE add-on.

There are more than a few ways to handle IPC on every OS. The ones that we implemented are *TCP stream* and *shared memory*. These are among the most common approaches available both on Windows and Linux. In either case, we must clearly define what the shared data represent. Although shared memory has more flexibility than TCP, we stuck to sending discrete text or binary messages of predetermined format over time, to keep the implementation simpler and the two approaches interchangable.

Another question is what will we encode in our message. At first we encoded the full state of the simulated Arduino in *snapshots* but due to its various shortcomings we later switched to sending discrete *events*. So the simulator in the backend and its interpratation in the frontend both operate in terms of events, but TCP and shared memory modules only understand the notion of sending and receiving messages with very distinct interfaces. That's why we need to design an adapter in both processes and for both IPC methods that sets up the mechanism and encodes or decodes the passing messages into structured event data the simulation logic understands. The IPC adapter then informs the other systems in it's process any time a new event is received.

1.2.2 Snapshotting

In the early iterations, a snapshot of the Arduino state was sent over one message. That included the pinout state, pin modes, timer, etc. The state was periodically requested by the client with a *Read* message without payload. On a change in the frontend, a *Write* message with the state as payload would be sent to the server and the values of input pins would be overwritten atomically to the backend's internal state.

This approach was easy to implement but flawed. The main issue was the frequency of reads. At lower frequencies, the system is prone to temporal aliasing. Find temporal aliasing definition. Some events, like very short impulses, can be missed completely. This is not much of an issue in purely combinational circuits. But we want our frontend to support stateful components, e.g. shift registers present on the Funshield, which are controlled by digital writes less than a microsecond a part.

Highering the frequency to such an extent is not viable. elaborate

1.2.3 Events

Right now synchronization between processes is maintained by recording every significant event produced by either the backend or the frontend, and sending it to the other process. Each process is equipped with an event queue, that consumes the events in order of the least recent to the most recent. Consuming an event means invoking the appropriate function based on the type of the event with arguments stored in the event's signature.

The Arduino object in the frontend has no logic on its own, so the only consumed events are those produced by the backend. For that reason a simple queue is sufficient, and no further synchronization is necessary. In the backend however, both simulator's and frontend's events are consumed. Sending an event to the frontend and receiving direct consequeunces of the event back can involve a significant time overhead. In the meantime the simulator produces more events that are consumed almost instantaneously. We need to ensure that the events are consumed in a valid order. We also need a robust queue implementation that isn't prone to race conditions.

An event is labeled with two timestamps. Timestamps don't have to be based on a real clock, the only requirement is that it is a non-decreasing sequence of numbers. The first timestamp sent is the simulation time, the time as perceived by the Arduino being simulated. That is time given by a clock that is started right before the `setup()` is invoked. It can be manipulated by the user by pausing or slowing down. This is to aid the user while analyzing the logs of the simulator.

The other timestamp included is *Local virtual time* (LVT)[1]. Each process holds its own integer value of LVT. This number is increased with newly produced events and events received from the other process. When a decision has to be made on which event to consume next (like in the case of the backend's queue), the event with lower LVT is chosen. The relation $LVT_1 > LVT_2$ means that the event with LVT_1 happens after the event with LVT_2. In other words, event 1 is the consequence of the event 2 regardless of the real time passed between the events.

The rest of the information stored in an event is its type and arguments. The type corresponds to the action taken after the event is consumed, like writing to a pin or changing pin mode. Each type can define up to 3 integer parameters, e.g for a *write* event that is id of the pin, and a digital value we want to write to it. By limiting the number of arguments, we intend to avoid heap allocation of these events, which could considerably slow down the simulation with programs that produce a lot of events in a short time.

There are many ways to encode such an event into a message. Currently the only encoder implemented in this work is the text encoder, which parses values of events into human readable strings separated by a colon. The messages themselves are separated by a semicolon. This particular encoding is quite space ineffective, but it helps a lot during debugging of the IPC systems.

1.2.4 TCP stream

Messages can be sent and received through a stream of data on a TCP connection. TCP is commonly used to send data over a network, meaning backend and frontend processes can run on different machines. But listening to a connection on the same machine on the same port works just as well. TCP messages are guaranteed to arrive in the same order as they were sent.

The backend implements the TCP server, while the frontend implements the client. The implementations and design considerations behind these are very different based on the language they were written. But in both cases TCP handlers need to run on their own threads, due to how heavyweight they are. There is a buffer to which read bytes are stored from an OS network stream. These are passed as a chunk to a higher level message handler that splits the chunk into individual messages according to predetermined delimeter character. When a message needs to be sent from another thread, it is passed to the message handler where it is posted to the end of a queue and is scheduled to be processed by the OS.

There is a minor limitation of IPC over TCP that makes the debugging experience of the user's code uncomfortable. As we established, hitting a breakpoint will block all running threads of the process, including the thread with the TCP handler. When stepping through the lines of the code where events are produced, the TCP thread has to schedule posting of the message with the event, and it takes time for it to be eventually sent. In many cases the TCP thread won't be fast enough to send the message in the time the main thread progresses to the next line where a breakpoint will be hit again. As a result, consequences of the event won't be reflected in the frontend process until another one or more lines have been stepped through. This is the main reason we decided to also implement IPC with memory mapping, even though TCP is fully functional.

[1]The terminology and principles are inspired by the *Optimistic synchronization* paradigm in distributed systems. In this paradigm, the events are consumed as fast as possible, and then rolled back once an event with a lower LVT than the ones consumed appears (so called *Time Warp* mechanism). But Time Warp is not viable for simulating code of a high level language like C++ without custom memory management, and in our implementation it isn't even necessary. [1]

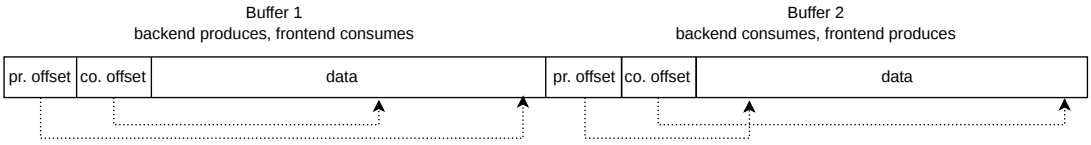

Figure 1.2 Memory layout of the shared memory region

1.2.5 Shared memory

Memory mapping is the functionality of an operating system to relate a region of main memory 1:1 to a region in the address space of a process. This is an important idea in establishing a virtual memory of a process, but OS objects can be mapped as well. It could be either a *memory-mapped file*, contents of a disk file moved to the RAM. Or it could be a special OS object called *shared memory*, living in the kernel heap, specifically designed for IPC. In both cases, multiple processes can map the same memory region to their own address space. Thus they have access to the same data.

The terminology and API differ significantly between Linux and Windows, but with the use of the right libraries, the differences can be abstracted away and used as a single concept. So from now on we refer to the idea of mapping the same memory region to multiple address spaces as *shared memory*.

This tool is very powerful, we can address the shared data by pointers as we would with any other data of the process. There is also no need for system calls once the the memory has been mapped. No special instructions need to be used for reading or writing and all changes are immediately visible on all of the processes that map the same region. As long as we do the writes in the simulating thread, we avoid the problem with delayed observations during debugging we encountered with TCP.

In the shared memory region, we establish a data structure that stores messages in a sequential order. We opted for a circular buffer to fully utilize the limited space of the region. There are actually two non-overlapping buffers in the same region together with a few bytes of metadata about their current state. A process consumes messages from one buffer while producing new messages to another. This is to avoid numerous race conditions that would occur while writing and reading from the same buffer while keep the synchronization mechanism lock-free.

As for the metadata, for each buffer a position after the last consumed and produced message is stored. It is important to never store references as pointers in the shared memory itself. The memory region will be most likely mapped to different addresses in each process. Because of that the *producer offset* and *consumer offset* are regular integers denoting the distance from the first adress of the buffer in bytes. When the offsets are equal, it means the buffer is empty. We don't care about concurrent updates of these offsets, one process changes only one of the offsets for the duration of the simulation.

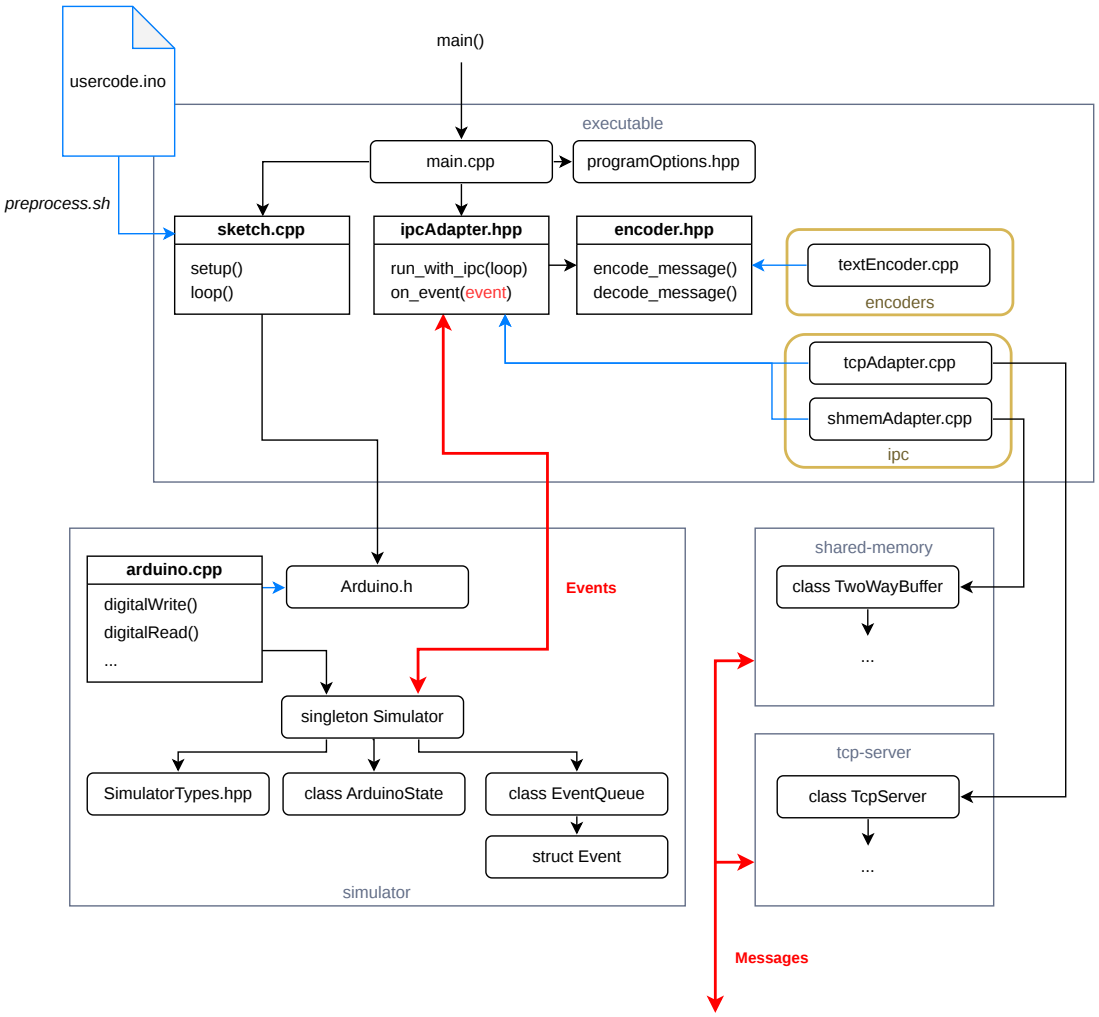

Figure 1.3 Backend architecture

1.3 Backend

1.3.1 The simulation

The language choice for backend was obvious, the `.ino` file is a syntactically valid C++ code. We include the code in the project by passing it as an argument to the `preprocess.sh` script (`preprocess.bat` on Windows). Inside we use the `arduino-cli` tool with the right arguments to apply very light prerpocessing to the file, like adding function declarations to the top or including the `Arduino.h` header. Important additions are the `#line` directives, which link to the definitions in the original `.ino` file. Thanks to these, breakpoints can be added to the `.ino` file and debugged as if the file itself was part of the project, completely with stepping through lines and inspection of variables. The file is then saved as `sketch.cpp` in the directory of the `executable` module where it is picked up by the CMake configuration and included in the `main.cpp`. The functions declared by the `Arduino.h` header file, like `digitalWrite` or `millis` are implemented in a custom mock library `Arduino.cpp` in the `simulator` module. Instead of instructing the AVR chip, as the real implementation would, the mock definitions create events and pass them to the `Simulator` for enquing and consumption.

The `Simulator` singleton class is the beating heart of the backend. It is responsible for handling the events and the event queue, managing the Arduino state, logging changes and keeping track of time. It is initialized in `main.cpp` and used frequently throughout the `executable` and `simulator` modules. The `Simulator` holds the instance of the `ArduinoState` class, representing the digital states on its pins, current pin modes and the inner timer of the Arduino. It also holds the instance of the `EventQueue`, which stores both local and remote events waiting for consumption. The `SimulatorTypes.hpp` is a header file with very few type defintions commonly used in the `simulator` module.

1.3.2 Events and IPC

The `Event` struct is just a "recipe" for what changes should the `Simulator` do to the `ArduinoState`. It holds a lambda function, corresponging to an event's type that is executed only after the event is put in the event queue and consumed in the order described in the section 1.2.3. The struct defines static functions that create new event instances with these predefined lambdas with the event type's parameters. When a local event is consumed, i.e. the lambda is executed, the event is signalled back to the `executable` module through a callback, that has been passed to the `Simulator` during initialization. This way the event can be sent to an IPC adapter, regardless of the used implementation, without a circular dependency on the `executable` module.

The IPC adapter itself is an interface of two functions declared in the `ipcAdapter.hpp`. `run_simulator_with_ipc` initializes the IPC mechanism. It takes the simulator loop function as an argument and executes it after IPC has been set up. That is because the IPC usually needs to create new threads that cannot be joined before the loop stops running. The loop function itself is defined in `main.cpp` and it contains both the `setup()` and `loop()` calls from the user's code.

The `on_event` function of the adapter is the callback passed to the `Simulator`. It encodes the event into a message and sends over IPC to the remote process. Listening to events of the remote process occurs on a different thread than the one with the simulator loop. Incoming messages are decoded into `Event` structs and sent directly to the event queue in the `Simulator`. `encode_message` and `decode_message` are functions declared in yet another interface in the `encoder.hpp` header. It's implementations are in the `encoders` directory, i.e. only `textEncoder.cpp` at the moment.

Implementations of the IPC adapter are located in the `ipc` directory. The `tcpAdapter.cpp` depends on the `tcp-server` module, while the `shmemAdatper.cpp` depends on the `shared-memory` module. Which one is used depends on the chosen compilation target. The `tcp-server` module contains definition of the TCP server and mechanisms for accepting new connections and handling the stream of data using the `boost::asio` library. The `shared-memory` module contains memory mapping mechanisms using the `boost::interprocess`, implementation of the circular buffer and the producer-consumer pattern using two buffers.

1.3.3 Utilities

Logger

There are additional utility modules not shown in the figure 1.3. One of them is the humble `logger` used extensively throughout the project. The `ILogger` abstract class exposes the `log` function accepting either a raw string or an `IMessage` abstract struct. Structs inheriting form `IMessage` represent unified formatting of a message with a timestamp and a level of verbosity. The verbosity levels from the highest to lowest are *Debug*, *Info*, *Warning* and *Error*. The messages of higher verbosity levels can be ommitted from the logs by setting the appropriate command line option. `ILogger` offers shorthand functions for logging these general messages, but more specific ones can be defined as well.

The module comes with `FileLogger` implementation of the `ILogger`. It creates or opens a file and dumps the logs inside. To avoid race conditions while writing to the files, two of these loggers are used in the project. One is owned and used by the `Simulator` singleton for logging inner workings of the Arduino. The other is owned by the IPC adapter and used for logging the exchanged messages or potential failures of the IPC mechanisms.

Timer

The other utility model is `Timer`. It is used to track time from the start of the simulation. The simulator holds two of these timers, one for *real time* and one for *simulation time*. Although there is no difference between them right now, the intention is to apply effects like stopping or slowing down to the simulation time timer from the frontend, so that functions like `millis` work consistently with disturbances caused by the frontend. Functions returning real time and simulation time are passed to the file logger as a source of its timestamps.

1.4 Frontend

1.4.1 The presentation

The frontend is written in C#. We consider the language to be more convenient for writing many of the essential systems than its C++ counterpart. That includes handling the graphics, the user interactions and parsing numerous configuration files needed for extensibility of the application. Libraries are also much easier to set up and compile thanks to NuGet.

For the GUI we decided to use Avalonia UI. Avalonia is a popular open-source cross-platform .NET framework, used by major companies like JetBrains or Unity. It is based on the Microsoft's WPF framework utilising the same XAML markup language and many of the same core principles. We chose it exactly because neither WPF or its cross-platform successor MAUI are available on Linux, one of our target platforms. We only include Avalonia in the Main project of the frontend, as this is the only project responsible for rendering.

Our humble GUI consists mostly of the circuit canvas. A zoomable and pannable space where the *components* of a circuit can be placed to specific coordinates, uncostrained by the usual layout structures of Avalonia. Arrangement

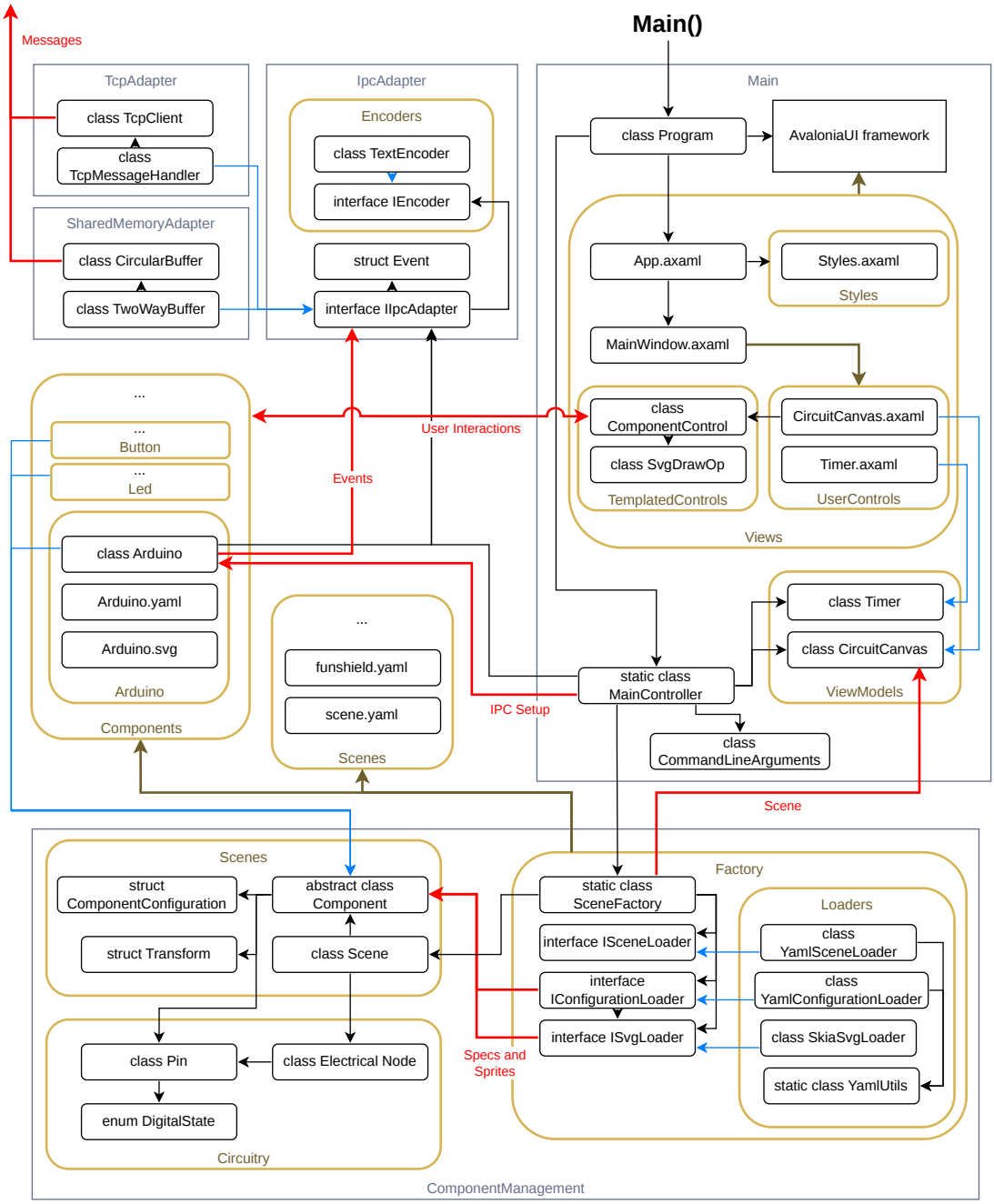

Figure 1.4 Frontend architecture

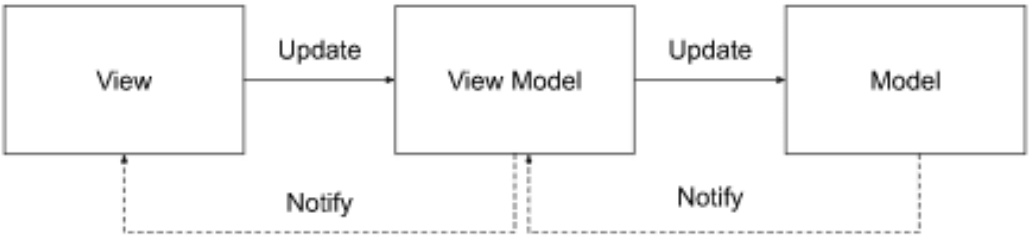

Figure 1.5 MVVM architecture [3]

of these components is called a *scene*. Each of the component types have its own set of SVG images that are drawn to the canvas. We refer to them as *sprites*. Each individual component always references one of its sprites, conveying the current state to the user. A sprite on the canvas can be clicked on (e.g. a button) to generate some kind of response. Of course, these features are not built into Avalonia, so we need to create custom Avalonia *controls* to achieve them.

Model-View-ViewModel

The preferred architecture for Avalonia applications is *MVVM*, or *Model-View-ViewModel*. *View* is the visual layout of the window and its parts. It is written in *XAML* (Extensible Application Markup Language) in files with the `.axaml` extension. In Avalonia's case, elements of a view are called *controls*. Those can be buttons, labels, containers, etc. 3 types of custom controls can be created in total, our frontend utilizes *user controls* and *custom-drawn controls*. User controls are themselves views built up in XAML from other controls. This is perfect for our `CircuitCanvas` view, as it is just a list of component controls enveloped in a built-in `Canvas` panel. The custom-drawn controls are defined as C# classes, inheriting from the `Control` class, and defining it's own rendering logic by overriding the `Render` method [2].

We need to implement the visuals of our components using custom-drawn controls, as there are no built-in or library controls that can draw a precompiled SVG image and swap it during runtime. Thus we define a `ComponentControl`, that calls a custom drawing operation in its `Render` method. That is defined by the `SvgDrawOp` class, which takes a sprite and a transformation matrix as arguments, applies visual transformations on the image and instructs the Avalonia's rendering pipeline to show it in the parent view.

XAML views bind to data and events in a *viewmodel*, which manipulates with data shown in the view. A model then is the raw data passed to the viewmodel, accessed by some database or a service, or produced by the business logic. View is aware of the viewmodel and viewmodel is aware of the model, but not vice versa. In ideal case, the model should be agnostic about the UI technology used. [3]

Our model is the abstract `Component` class implemented by various types of components, like LEDs, buttons or our Arduino. A list of the `Component` instances is stored in `CircuitCanvas` viewmodel, and each instance is passed to a `ComponentControl` in the `CircuitCanvas` view. This and all other initialization steps of the application are orchestrated by the static `MainController` class.

1.4.2 The logic

Only a minor part of the frontend is actually written with Avalonia. The business logic of the aforemention `Component` model is all encompassed in the `ComponentManagement` project. That means parsing of the components definition, creating instances of a scene, and the simulation of the circuit. This project is completely unaware, that it will be rendered with Avalonia specifically.

The circuit can be thought of as an unoriented multigraph, where the nodes of the graph are both *electrical nodes* and the *components*. All neighbors of an electrical node are components, and all neighbors of a component are electrical nodes. An edge of the graph represents a conductive connection between an electrical node and a *pin* of a component. Components usually have multiple pins.

In a digital circuit, an electrical node always obtains one of two digital states, *high* or *low*, which is given by states of pins it is connected to. When a high state is written into a pin, we say it is *driving*. When a low state is written into a pin, it becomes not driving again. The digital state of an electrical node is high precisely when at least one of the pins it is connected to is driving. A change in the state of an electrical node is propagated through all of the connected pins. Components can react to this change by writing new states to their other pins creating a chain reaction of state changes in the circuit.

Components

The frontend's user is encouraged to extend the simulation with their own implementations of the abstract `Component` class. That is done by creating a new *component directory* with the name of the component. The shared path to all component diretories is specified as one of the command line options of the frontend. The static information about the component type is stored as a record `ComponentConfiguration`, created from the files in the directory. The required files are a YAML file with metadata and at least one SVG file for the sprite. The YAML file contains in the very least a name to identify the component type, and a list of the component's pins. The pins can be defined either as an enumeration of their names, or as a single integer, that represents the total count of unnamed pins identified by their index.

The `Component` base class provides multiple virtual methods with the `On*` prefix. These are called upon changes in the circuit or as a result of a user interaction. The most important one is the `OnPinStateChanged` with the `Pin` object passed as an argument. This is the primary way of defining the component's logic. `OnControlPress` and `OnControlRelease` are other useful methods for interactible components, such as buttons. The base method `UpdateSprite` changes the rendered sprite to a different one, identified by the name of its SVG file. The base `GetPin` method returns a `Pin` object of the component's instance identified either by its name or index. Digital states are read and written through this object.

Usually, specific pins have inherent semantics of being used only for reading or only for writing. This property can be set explicitly on the `Pin` object with `MakeReadOnly` and `MakeWriteOnly` methods. Read only pins refuse write attempts from the component's logic and write only pins refuse notifying a component with a new state from node's propagation. General pins, defined by method `MakeGeneral`

don't have these constraints. Incorrect use of pins with these semantics is followed by a warning in the simulation logs.

Listing 1.1 Component configuration example: Led.yaml

```yaml
name: Led
description: Light emitting diode
imageSize: 50 50
pinNames:
  - anode
  - cathode
```

Scenes

The final form of the graph is achieved with creation of the scene. The YAML file of the scene consists of two sections. In the `Components` section, individual instances of `Component` subtypes are defined. They must have a unique name, and they should specify a graphic transformation, i.e. position, rotation and scale. This triplet of 2D vectors is composed into a `Transform` struct and passed into the `SvgDrawOp` as a transformation matrix. The `Nodes` section defines individual electrical nodes. A single node in this file is represented as a list of component-pin pairs, where the components are identified by their instance name in the `Components` section, and the pin is identified by its definition in the `ComponentConfiguration` of the given `Component` subtype.

The creation of the scene is comprised of a series of parsers in the `Factory` directory. There is an interface for each of the file types, i.e. the scene, component configurations and SVG files. Each has exactly one implementation in the `Loaders` directory, using `YamlDotNet` and `SvgSkia` libraries. They are put together in the static `SceneFactory` class, where component subtypes and instances are finalized. The results is a complete `Scene` object with initialized `Component` instances and `ElectricalNode` lists. The `SceneFactory` is called by the `MainController` in the `Main` project, which passes the scene to the `CircuitCanvas` viewmodel.

Listing 1.2 Scene example: minimal.yaml

```yaml
Components:
  Led:
    Led1:
      Position: 100 100
    Led2:
      Position: 200 100
  Button:
    Btn:
      Position: 100 200
  Vcc:
    Source:
      Position: 125 300
Nodes:
  - [Source.0, Btn.0]
  - [Led1.cathode, Btn.1]
  - [Led2.cathode, Btn.1]
```

The example scene in the listing 1.2 contains 2 LEDs, 1 button and 1 source of voltage. Source, which has one pin that is always driving, is connected to the 0th pin of the button, which reads a high value of the node. Both LEDs are connected

to the 1st pin of the same button. Pressing the button will copy the digital state from pin 0 to pin 1, and thus lights up the LEDs.

1.4.3 The Arduino component and IPC

Arduino is one of the component subtypes of the `ComponentManagement` project. It is a partial class divided into 2 files. In `Arduino.cs` it overrides virtual methods and deals with interaction with the circuit. In `Arduino.Events.cs` it creates events and sends to IPC. It also calls its own method from `Arduino.cs` when a new event arrives, based on its type and arguments. The adapter is now defined as an `IIpcAdapter` interface, which is owned by the Arduino component. Events are exchanged through this interface.

The interface is defined in the `IpcAdapter`, and is implemented by the `TcpAdapter` and `SharedMemoryAdapter` projects. It declares methods for connecting to IPC and sending messages. It also declares a C# event called `ReceivedEventEvent`, which is invoked after decoding a message incoming from the backend, with the decoded `Event` struct as an argument. The Arduino component is subscribed to `ReceivedEventEvent`.

The `IpcAdapter` project also contains message encoders and a definition of the `Event` struct. The C++ and the C# version of the `Event` struct have the same signature, with the exception of owning a lambda function (described in 1.3.2). The lambda is not necessary for the frontend, as there will be only one source of events executed in the frontend, the ones received by the backend. They don't have to be put in a queue, they can be consumed immediately as they come. Frontend also produces events, e.g. when value on an Arduino's input pin is changed from the circuit, but those are never consumed by the frontend. It is the reponsibility of the backend to handle them.

`IEncoder` and its `TextEncoder` implementation are analogous to `encoder.hpp` and `textEncoder.cpp` in the backend. The interface declares `EncodeEvent` and `TryDecodeEvent`, which transcribe events structs into human-readable strings and vice versa.

1.4.4 Utilities

Logger

The `Logger` project is analogous to its backend counterpart (1.3.3). It provides logging to a file with 4 levels of verbosity, that can be set up with a command line argument. Two loggers are used in the frontend as well, one for IPC and one for circuitry logs. There is one additional implemenation of the `ILogger` called `CompositeLogger`, which holds list of loggers, and logs into all of them simultaneously. This mechanism anticipates a need for another logger implementation, which will display the log messages in the GUI using Avalonia.

2 Implementation

2.1 Backend

2.1.1 Simulator module

2.1.2 Executable module

2.1.3 TCP Adapter module

2.1.4 Shared Memory Adapter module

2.1.5 Utilites

2.2 Frontend

2.2.1 Component Management project

2.2.2 Main project with Avalonia UI

2.2.3 IPC Adapter project

2.2.4 TCP Adapter project

2.2.5 Shared Memory Adapter project

2.2.6 Logger project

3 Developer documentation

3.1 Compiling and running Arduino code

3.2 Defining components

3.3 Defining scenes

3.4 IPC interface

Conclusion

Bibliography

1. FUJIMOTO, R.M. Parallel and distributed simulation systems. In: *Proceeding of the 2001 Winter Simulation Conference (Cat. No.01CH37304)* [online]. Arlington, VA, USA: IEEE, 2001, pp. 147–157 [visited on 2026-03-05]. ISBN 978-0-7803-7307-5. Available from DOI: 10.1109/WSC.2001.977259.
2. *Choosing a custom control type | Avalonia Docs* [online]. 2026-04-20. [visited on 2026-05-19]. Available from: https://docs.avaloniaui.net/docs/custom-controls/choosing-a-custom-control-type.
3. *The MVVM pattern | Avalonia Docs* [online]. 2026-04-20. [visited on 2026-05-19]. Available from: https://docs.avaloniaui.net/docs/fundamentals/the-mvvm-pattern.

List of Figures

List of Tables

List of Abbreviations

A Attachments

A.1 First Attachment